Guide technique

Architecture, modèles de données et extension

Module **restaurant** — Tryton 7.0 — version 7.0.1

1. Vue d'ensemble

Le module **restaurant** (paquet pip **restaurant-tryton**) ajoute à Tryton 7.0 la gestion des restaurants et bars : salle et tables, commandes (sur place, ardoise de bar, à emporter, livraison), écrans cuisine et bar, réservations, menu avec recettes, happy hours, paiements, livreurs, consommation de stock, tableau de bord et rapport de ventes.

Dépendances : **company, currency, ir, party, product, res, stock**. Aucun module de vente ni de comptabilité n'est requis ; les écritures comptables et factures ne sont pas générées (voir limites).

2. Modèles de données

Modèle	Rôle
restaurant.restaurant	Établissement : code, société, entrepôt, taux de service, taxe incluse, frais et minimum de livraison, remise maximale.
restaurant.area / restaurant.table	Zones et tables ; le statut libre/occupé est calculé à partir des commandes ouvertes (champ Function avec recherche).
product.template (extension)	restaurant_item, course, prep_station (cuisine/bar), sold_out, track_stock, recipe.
restaurant.recipe / .recipe.line	Une recette par article ; ingrédients en quantité par portion (3 décimales).
restaurant.happy_hour	Remise (%) par jour, plage horaire, catégorie et restaurant optionnels.
restaurant.order	Commande (workflow open → paid / cancelled), totaux calculés, suivi de livraison.
restaurant.order.line	Article : états new → sent → preparing → ready → served (ou cancelled).
restaurant.payment	Espèces, carte, mobile money, bon, sur compte ; montant remis et monnaie rendue.
restaurant.reservation	Réservation (booked, seated, cancelled, no_show) avec contrôle de chevauchement.
restaurant.driver	Livreur, véhicule, nombre de livraisons en cours.
restaurant.report.sales	Vue SQL (table_query) des ventes payées par jour, type et produit.
restaurant.dashboard	Vue SQL par restaurant avec compteurs calculés.

3. Règles métier

- **Table occupée** : une seule commande ouverte par table (validation à la création et à l'écriture).
- **Totaux** : sous-total – remise + service (sur place et bar uniquement) + frais de livraison (gratuits au-delà du seuil). La taxe est incluse dans les prix et affichée à titre informatif.
- **Remise** : au-delà du plafond du restaurant, le groupe Responsable est requis.
- **Envoi** : refuse les articles épuisés ; avertit si le nombre de couverts dépasse les places (res.user.warning).
- **Livraison** : minimum, adresse et téléphone obligatoires ; assigner un livreur passe l'état à « assignée » ; expédier puis livré ou échoué.
- **Paiement** : le solde (total + pourboire – payé) doit être nul pour payer ; la commande payée consomme le stock.
- **Stock** : ingrédients de la recette (ou l'article lui-même si track_stock) déplacés de l'emplacement de stockage vers l'emplacement client, origine = commande. Les quantités sont arrondies à l'unité de

mesure : utilisez une unité fractionnaire (kg, litre) pour les ingrédients.

- **Happy hour** : à l'ajout d'un article, la meilleure remise applicable est copiée sur la ligne.

4. Droits et sécurité

Groupe	Droits
Restaurant	Service : commandes, paiements, réservations (lecture/écriture/création).
Cuisine et bar	Lecture des commandes ; modification de l'état des articles de son poste.
Responsable	Remises au-delà du plafond, annulation d'articles déjà envoyés.
Administration	Configuration : restaurants, tables, livreurs, recettes, happy hours.

Des règles ir.rule limitent chaque modèle aux sociétés de l'utilisateur. Les boutons du workflow sont déclarés dans ir.model.button.

5. Extension

- Héritez de **restaurant.order** avec PoolMeta pour ajouter des champs ; surchargez *_consume_stock()* pour changer la sortie de stock.
- Pour la facturation, créez un module compagnon qui génère une facture brouillon à partir de la commande payée (modèle : fuel_station_invoice).
- Le ticket est un modèle ODF (receipt.fodt) rendu par relatorio ; modifiez-le avec LibreOffice.

6. Modules compagnons

Module	Contenu
restaurant_invoice	Crée une facture client brouillon à partir d'une commande payée avec client (articles, service, frais de livraison) ; paramètre « Facturation » du restaurant.
sale_point_cash_control	Étend sale_point : coupures de billets et pièces (sale.point.cash.denomination), comptage à l'ouverture et à la fermeture de la session de caisse (sale.point.cash.count), billets suspects exclus, écart enregistré avec un type de transfert, avertissement si l'ouverture diffère de la fermeture précédente.
sale_point_quick	Étend sale_point : saisie rapide par code (3*COLA), produits favoris, boutons Payer en espèces / par carte en un clic.
party_person_ht	Langue créole haïtien (ht) dans ir.lang, prénom des personnes, identifiants NIF et NINU, types de subdivision arrondissement et section communale, traduction ht.po de ses chaînes et de celles de party/country.

Le module restaurant intègre aussi sa propre caisse (restaurant.cash.session / .count / .denomination) pour les paiements en espèces, avec le même principe de comptage billets et pièces.

7. Aide, couleurs et avertissements

- Tous les champs des modules fournissent une bulle d'aide (attribut help) visible au survol dans le client.
- Les listes utilisent des couleurs (visual) : table occupée en orange/libre en vert, article en préparation en orange, écart de caisse en rouge.

- Les avertissements (res.user.warning) préviennent sans bloquer : couverts supérieurs aux places, écart de caisse à la fermeture, ouverture différente de la fermeture précédente, billets suspects.
- Les traductions fr.po et ht.po sont fournies pour chaque module.

8. Tests

Plus de 200 tests au total sur la suite (restaurant 69, restaurant_invoice 23, sale_point_cash_control 28, sale_point_quick 22, party_person_ht 51) : tests génériques Tryton (vues, droits, accès, règles) et tests fonctionnels (table occupée, totaux, plafond de remise, flux cuisine, article épuisé, paiement et stock, livraison, réservations, happy hour, rapports, guide).

```
DB_NAME=:memory: python -m unittest trytond.modules.restaurant.tests.test_module
```

Restaurant : architecture des liaisons

Mise à jour v3 — ajout au guide d'origine. Auteur : Pierre Michel Augustin — AGPLv3.

Module	Rôle	Points d'extension
restaurant_mobile_payment	Bouton Payer par mobile, méthode « mobile »	hooks de l'assistant mobile_payment
hotel_restaurant	Commande liée au séjour, méthode « room »	restaurant.order, restaurant.payment, restaurant.reservation
restaurant_invoice	Facturation des commandes	les commandes imputées à la chambre ne sont pas facturées ici

La caisse du restaurant réutilise `CountMixin` et les coupures de sale_point_cash_control : aucune table de coupures propre. Test ajouté : ouverture différente de la dernière fermeture (avertissement à confirmer).